Lab: time series objects in R

Contents

Representing regularly spaced data as ts/mts objects

Time series in R can be represented with an object of class `ts`. We shall learn how `ts` works with an artificial time series simulated from the model

$$X_t = 10 + \frac{1}{4}t + \epsilon_t, \quad t = 1, ..., 20$$

where ϵ_t is a standard normal white noise process. In R we can simulate a realization of the above model $x_1, ..., x_{20}$ with the following commands:

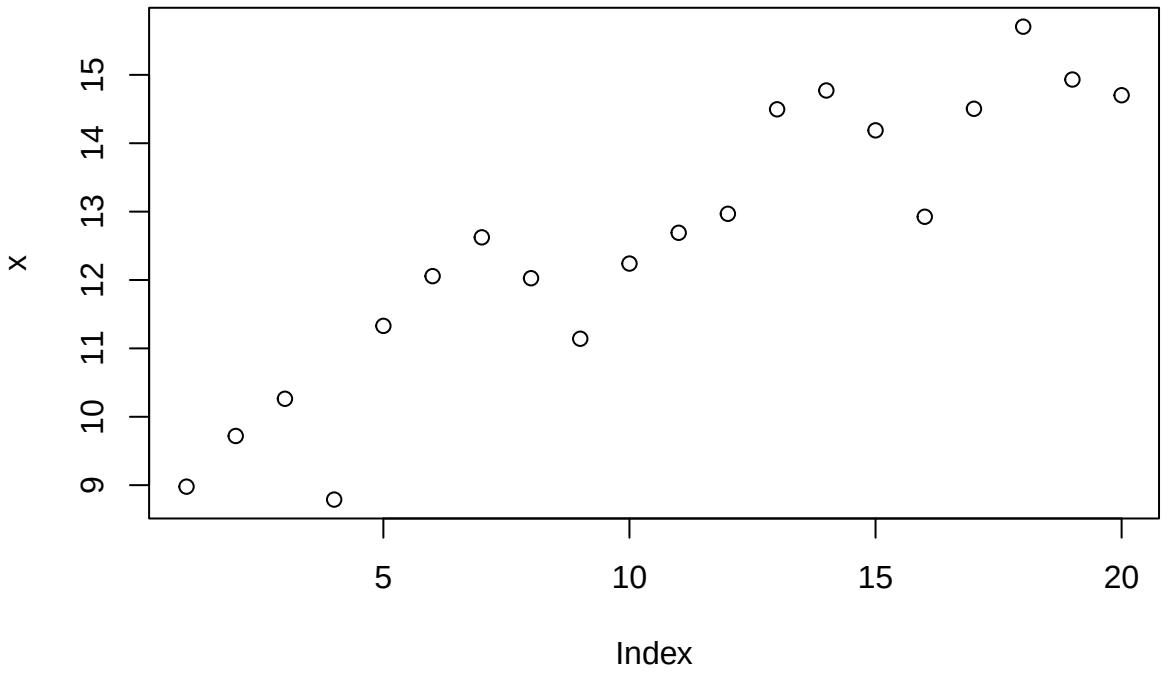
```
tm <- 1:20 ## set "time"
tm
```

```
##  [1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

```
set.seed(318) ## random generator seed
x <- 10 + 0.25 * tm + rnorm(20) ## simulated series
x
```

```
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

```
plot(x)
```




```
y <- ts(x) ## specify the `ts' class
y
```

```
## Time Series:
## Start = 1
## End = 20
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

The vector `y` is an object of class `ts`:

```
class(y)
```

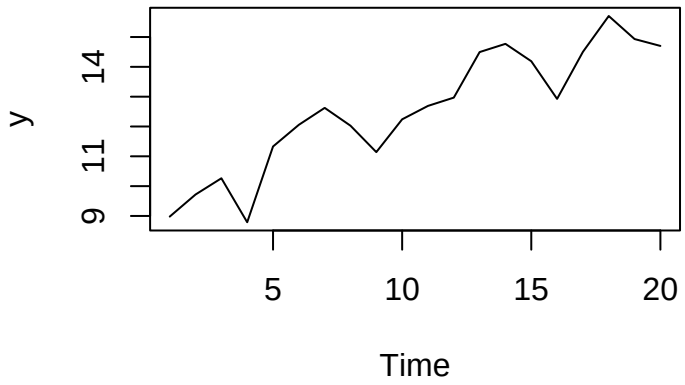
```
## [1] "ts"
```

```
is.ts(y)
```

```
## [1] TRUE
```

Plot of the time series:

```
plot(y)
```



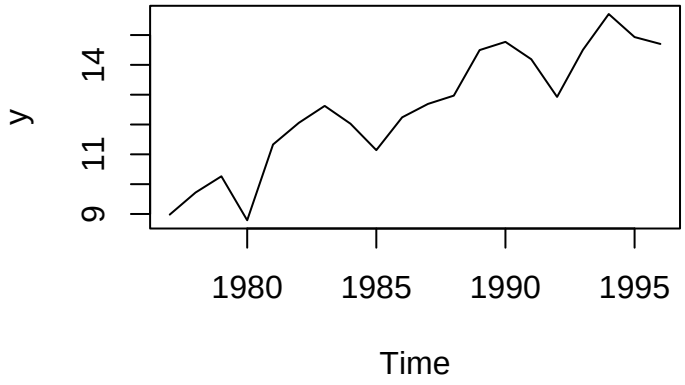
Now, suppose that the artificial time series has an annual frequency and that it starts from 1977:

```
y <- ts(x, start = 1977)
y
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

The plotted series becomes:

```
plot(y)
```



We can also set the end of the series:

```
ts(x, end = 1996)
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

or both `start` and `end`:

```
ts(x, start = 1977, end = 1996)
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

```
ts(x, start = 1977, end = 1986)
```

```
## Time Series:
## Start = 1977
## End = 1986
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634
```

```
ts(x, start = 1977, end = 2003)
```

```
## Time Series:
## Start = 1977
## End = 2003
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857  8.978189
## [22]  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
```

Note the dangerous behaviour of R when the observations are not enough to complete the series (third example): The observations are recycled!

First operations on time series

It is possible to modify singular or groups of observations:

```
z <- y
z[3] <- 0
z
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088  0.000000  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

```
z[z > 14] <- 5
z
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088  0.000000  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200  5.000000  5.000000
## [15]  5.000000 12.925156  5.000000  5.000000  5.000000  5.000000
```

```
z[c(7, 12)] <- -100
z
```

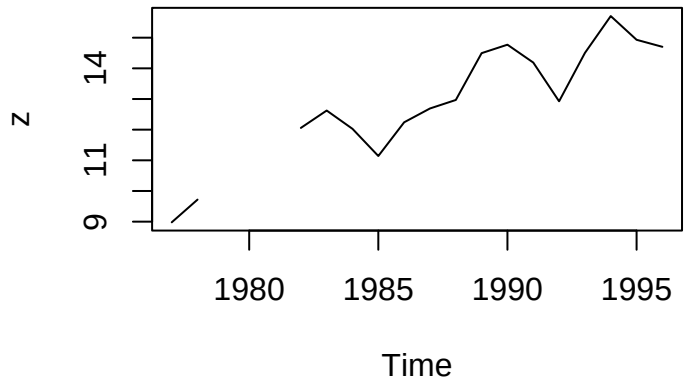
```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]    8.978189    9.719088    0.000000    8.789285   11.329231   12.056625
##  [7] -100.000000   12.026296   11.141061   12.240634   12.690619 -100.000000
## [13]    5.000000    5.000000    5.000000   12.925156    5.000000    5.000000
## [19]    5.000000    5.000000
```

Missing values are denoted with `NA` (not available):

```
z <- y
z[3:5] <- NA
z
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088        NA        NA        NA 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

```
plot(z)
```



Functions can be applied to a time series of class `ts`:

```
10 * y + 100
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1] 189.7819 197.1909 202.6308 187.8928 213.2923 220.5663 226.2391 220.2630
##  [9] 211.4106 222.4063 226.9062 229.6820 244.9615 247.7129 241.8858 229.2516
## [17] 245.0382 257.0444 249.3185 247.0186
```

```
log(y)
```

```
## Time Series:
## Start = 1977
## End = 1996
```

```
## Frequency = 1
##  [1] 2.194798 2.274092 2.328553 2.173533 2.427386 2.489614 2.535592 2.487096
##  [9] 2.410637 2.504761 2.540863 2.562500 2.673883 2.692685 2.652437 2.559175
## [17] 2.674412 2.753943 2.703497 2.687974
```

```
mean(y)
```

```
## [1] 12.55247
```

```
var(y)
```

```
## [1] 4.213293
```

```
length(y)
```

```
## [1] 20
```

```
range(y)
```

```
## [1]  8.789285 15.704436
```

```
c(min(y), max(y))
```

```
## [1]  8.789285 15.704436
```

```
which(y == max(y))
```

```
## [1] 18
```

Handling time

Function `time` returns the measurement times (*i.e.*, the times at which observations are sampled):

```
time(y)
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1] 1977 1978 1979 1980 1981 1982 1983 1984 1985 1986 1987 1988 1989 1990 1991
## [16] 1992 1993 1994 1995 1996
```

Function `window` allows to extract subseries of observations:

```
window(y, start = 1990)
```

```
## Time Series:
## Start = 1990
## End = 1996
## Frequency = 1
## [1] 14.77129 14.18858 12.92516 14.50382 15.70444 14.93185 14.70186
```

```
window(y, end = 1983)
```

```
## Time Series:
## Start = 1977
## End = 1983
## Frequency = 1
## [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
```

```
window(y, start = 1984, end = 1986)
```

```
## Time Series:
## Start = 1984
## End = 1986
## Frequency = 1
## [1] 12.02630 11.14106 12.24063
```

```
window(y, start = 1970, end = 2003)
```

```
## Warning in window.default(x, ...): 'start' value not changed
```

```
## Warning in window.default(x, ...): 'end' value not changed
```

```
## Time Series:
## Start = 1977
## End = 1996
## Frequency = 1
##  [1]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
##  [8] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [15] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857
```

```
window(y, start = 1970, end = 2003, extend = TRUE)
```

```
## Time Series:
## Start = 1970
## End = 2003
## Frequency = 1
##  [1]        NA        NA        NA        NA        NA        NA        NA
##  [8]  8.978189  9.719088 10.263081  8.789285 11.329231 12.056625 12.623907
## [15] 12.026296 11.141061 12.240634 12.690619 12.968200 14.496155 14.771287
## [22] 14.188576 12.925156 14.503824 15.704436 14.931851 14.701857        NA
## [29]        NA        NA        NA        NA        NA        NA
```

The last two examples show that `window` does not extend the series to fulfill the window dimensions, except when option `extend = TRUE` is given.

Seasonal time series

We consider now another artificial time series simulated from model

$$Y_t = 5\cos(2\pi t/12) + \epsilon_t, \quad t = 1, ..., 60$$

where again ϵ_t is a standard normal white noise. We further assume that measurement times correspond to months since 1977. Next lines simulate a realization from the above model:

```
tm <- 1:60
x <- 5 * cos(2*pi*tm/12) + rnorm(60)
```

Since this is a monthly time series, we need to set this information in `ts` using argument `frequency`:

```
ym <- ts(x, start = 1977, frequency = 12) ## frequency = 12!
ym
```

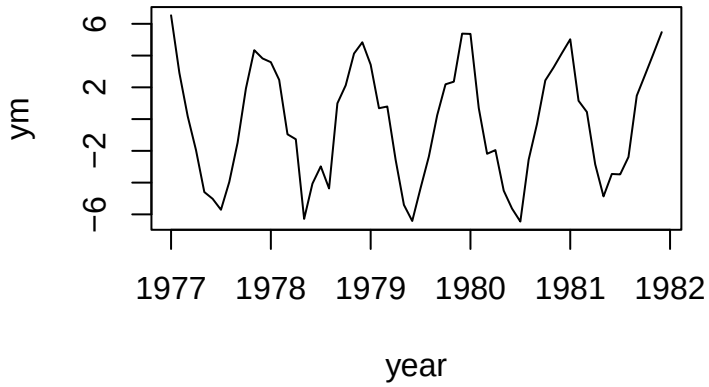
```
##             Jan        Feb        Mar        Apr        May        Jun
## 1977  6.5347134  2.9067153  0.1776022 -1.9589346 -4.5938283 -5.0188418
## 1978  3.5837494  2.4705428 -0.9571758 -1.2678985 -6.2828917 -4.0643132
## 1979  3.4263713  0.6846618  0.7918158 -2.5518682 -5.4015798 -6.4164253
```

```
## 1980  5.3625212  0.6923756 -2.1847280 -1.9536713 -4.5061232 -5.6346036
## 1981  5.0253054  1.1536394  0.4477242 -2.8477640 -4.8686560 -3.4561162
##             Jul        Aug        Sep        Oct        Nov        Dec
## 1977 -5.7099148 -3.9600899 -1.4854199  1.9259035  4.3400938  3.8180595
## 1978 -2.9766020 -4.3674311  0.9915215  2.1271179  4.1271216  4.8369648
## 1979 -4.3471811 -2.3524602  0.2453717  2.1905677  2.3533452  5.3814883
## 1980 -6.4507366 -2.5538876 -0.3085930  2.4366772  3.2551243  4.1560221
## 1981 -3.4824191 -2.3866368  1.4730737  2.7877591  4.1174645  5.4743698
```

```
time(ym)
```

```
##           Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1977 1977.000 1977.083 1977.167 1977.250 1977.333 1977.417 1977.500 1977.583
## 1978 1978.000 1978.083 1978.167 1978.250 1978.333 1978.417 1978.500 1978.583
## 1979 1979.000 1979.083 1979.167 1979.250 1979.333 1979.417 1979.500 1979.583
## 1980 1980.000 1980.083 1980.167 1980.250 1980.333 1980.417 1980.500 1980.583
## 1981 1981.000 1981.083 1981.167 1981.250 1981.333 1981.417 1981.500 1981.583
##           Sep      Oct      Nov      Dec
## 1977 1977.667 1977.750 1977.833 1977.917
## 1978 1978.667 1978.750 1978.833 1978.917
## 1979 1979.667 1979.750 1979.833 1979.917
## 1980 1980.667 1980.750 1980.833 1980.917
## 1981 1981.667 1981.750 1981.833 1981.917
```

```
plot(ym, xlab = 'year')
```



R has automatically assumed that the first observation corresponds to January of the starting year. We can modify the starting year through option `start`:

```
ym <- ts(x, start = c(1977, 5), frequency = 12) ## start in May 1977
ym
```

```
##             Jan        Feb        Mar        Apr        May        Jun
## 1977                                                6.5347134  2.9067153
## 1978 -1.4854199  1.9259035  4.3400938  3.8180595  3.5837494  2.4705428
## 1979  0.9915215  2.1271179  4.1271216  4.8369648  3.4263713  0.6846618
## 1980  0.2453717  2.1905677  2.3533452  5.3814883  5.3625212  0.6923756
## 1981 -0.3085930  2.4366772  3.2551243  4.1560221  5.0253054  1.1536394
## 1982  1.4730737  2.7877591  4.1174645  5.4743698
##             Jul        Aug        Sep        Oct        Nov        Dec
## 1977  0.1776022 -1.9589346 -4.5938283 -5.0188418 -5.7099148 -3.9600899
## 1978 -0.9571758 -1.2678985 -6.2828917 -4.0643132 -2.9766020 -4.3674311
## 1979  0.7918158 -2.5518682 -5.4015798 -6.4164253 -4.3471811 -2.3524602
## 1980 -2.1847280 -1.9536713 -4.5061232 -5.6346036 -6.4507366 -2.5538876
```

```
## 1981  0.4477242 -2.8477640 -4.8686560 -3.4561162 -3.4824191 -2.3866368
## 1982
```

```
time(ym)
```

```
##           Jan      Feb      Mar      Apr      May      Jun      Jul      Aug
## 1977                                     1977.333 1977.417 1977.500 1977.583
## 1978 1978.000 1978.083 1978.167 1978.250 1978.333 1978.417 1978.500 1978.583
## 1979 1979.000 1979.083 1979.167 1979.250 1979.333 1979.417 1979.500 1979.583
## 1980 1980.000 1980.083 1980.167 1980.250 1980.333 1980.417 1980.500 1980.583
## 1981 1981.000 1981.083 1981.167 1981.250 1981.333 1981.417 1981.500 1981.583
## 1982 1982.000 1982.083 1982.167 1982.250
##           Sep      Oct      Nov      Dec
## 1977 1977.667 1977.750 1977.833 1977.917
## 1978 1978.667 1978.750 1978.833 1978.917
## 1979 1979.667 1979.750 1979.833 1979.917
## 1980 1980.667 1980.750 1980.833 1980.917
## 1981 1981.667 1981.750 1981.833 1981.917
## 1982
```

Consistently, functions `start` and `end` return both the year and the month:

```
start(ym)
```

```
## [1] 1977    5
```

```
end(ym)
```

```
## [1] 1982    4
```

Behavior of `window` with the monthly series:

```
window(ym, start = 1978)
```

```
##             Jan        Feb        Mar        Apr        May        Jun
## 1978 -1.4854199  1.9259035  4.3400938  3.8180595  3.5837494  2.4705428
## 1979  0.9915215  2.1271179  4.1271216  4.8369648  3.4263713  0.6846618
## 1980  0.2453717  2.1905677  2.3533452  5.3814883  5.3625212  0.6923756
## 1981 -0.3085930  2.4366772  3.2551243  4.1560221  5.0253054  1.1536394
## 1982  1.4730737  2.7877591  4.1174645  5.4743698
##             Jul        Aug        Sep        Oct        Nov        Dec
## 1978 -0.9571758 -1.2678985 -6.2828917 -4.0643132 -2.9766020 -4.3674311
## 1979  0.7918158 -2.5518682 -5.4015798 -6.4164253 -4.3471811 -2.3524602
## 1980 -2.1847280 -1.9536713 -4.5061232 -5.6346036 -6.4507366 -2.5538876
## 1981  0.4477242 -2.8477640 -4.8686560 -3.4561162 -3.4824191 -2.3866368
## 1982
```

```
window(ym, start = c(1978, 9))
```

```
##             Jan        Feb        Mar        Apr        May        Jun
## 1978
## 1979  0.9915215  2.1271179  4.1271216  4.8369648  3.4263713  0.6846618
## 1980  0.2453717  2.1905677  2.3533452  5.3814883  5.3625212  0.6923756
## 1981 -0.3085930  2.4366772  3.2551243  4.1560221  5.0253054  1.1536394
## 1982  1.4730737  2.7877591  4.1174645  5.4743698
##             Jul        Aug        Sep        Oct        Nov        Dec
## 1978                       -6.2828917 -4.0643132 -2.9766020 -4.3674311
## 1979  0.7918158 -2.5518682 -5.4015798 -6.4164253 -4.3471811 -2.3524602
## 1980 -2.1847280 -1.9536713 -4.5061232 -5.6346036 -6.4507366 -2.5538876
```

```
## 1981  0.4477242 -2.8477640 -4.8686560 -3.4561162 -3.4824191 -2.3866368
## 1982
```

```
window(ym, end = c(1978, 9))
```

```
##             Jan        Feb        Mar        Apr        May        Jun
## 1977                                                6.5347134  2.9067153
## 1978 -1.4854199  1.9259035  4.3400938  3.8180595  3.5837494  2.4705428
##             Jul        Aug        Sep        Oct        Nov        Dec
## 1977  0.1776022 -1.9589346 -4.5938283 -5.0188418 -5.7099148 -3.9600899
## 1978 -0.9571758 -1.2678985 -6.2828917
```

```
window(ym, start = c(1978, 5), end = c(1978, 9))
```

```
##             May        Jun        Jul        Aug        Sep
## 1978  3.5837494  2.4705428 -0.9571758 -1.2678985 -6.2828917
```

Finally, to extract all the July observations of `ym`, we can use the argument `deltat = 1`:

```
window(ym, start = c(1977, 7), deltat = 1)
```

```
## Time Series:
## Start = 1977.5
## End = 1981.5
## Frequency = 1
## [1]  0.1776022 -0.9571758  0.7918158 -2.1847280  0.4477242
```

Manipulating ts objects

Some common manipulations of time series data involve lags and differences using the functions `lag()` and `diff()`. For example, to lag `ym` by one time period use

```
lag(ym)
```

To lag the price data by 12 periods use

```
lag(ym,k =12)
```

The `lag()` function shifts the time index back by an amount k. To shift the time index forward set k to a negative number

```
lag(ym,k =-1)
lag(ym,k= -12)
```

To compute the first difference in prices use

```
diff(ym)
```

Notice that application of `diff` is equivalent to

```
ym-lag(ym,k=-1)
```

To compute a 12 lag difference (annual difference for monthly data) use

```
diff(ym,lag=12)
```

which is equivalent to using

```
ym-lag(ym,k=-12)
```

Yearly time series

Read the global warning temperature data from file `globtemp.txt`:

```
temp <- scan("globtemp.txt")
```

First look at the data:

```
head(temp)
```

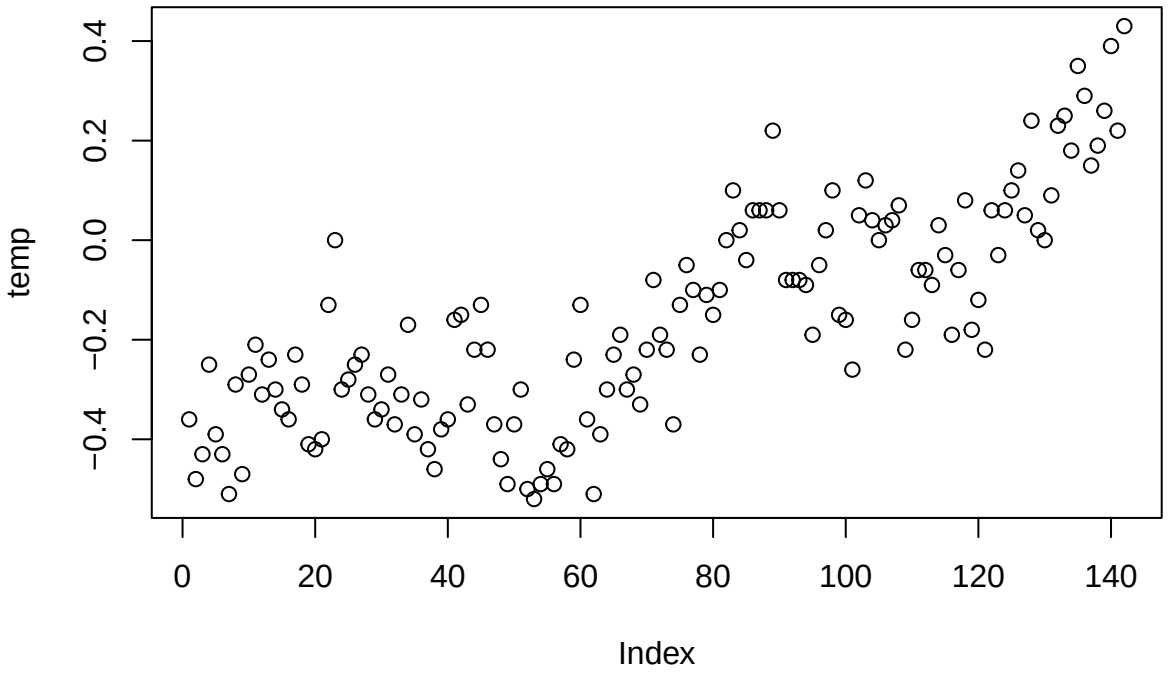
```
## [1] -0.36 -0.48 -0.43 -0.25 -0.39 -0.43
```

```
summary(temp)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -0.5200 -0.3300 -0.1900 -0.1581  0.0150  0.4300
```

Scatterplot:

```
plot(temp)
```



Transform `temp` to an object of class `ts`:

```
temp.ts <- ts(temp, start = 1856, frequency = 1)
class(temp)
```

```
## [1] "numeric"
```

```
class(temp.ts)
```

```
## [1] "ts"
```

```
head(time(temp))
```

```
## [1] 1 2 3 4 5 6
```

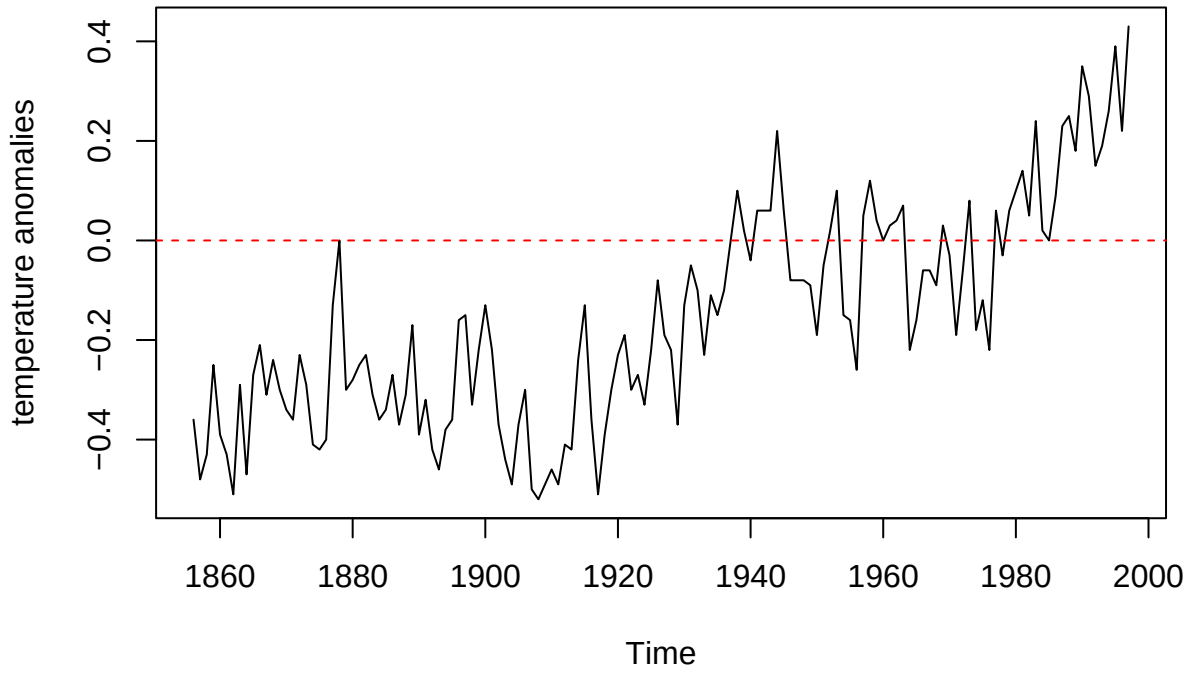
```
head(time(temp.ts))
```

```
## Time Series:
```

```
## Start = 1856
## End = 1861
## Frequency = 1
## [1] 1856 1857 1858 1859 1860 1861
```

Plot the series:

```
plot(temp.ts, ylab = "temperature anomalies")
abline(h = 0, col = "red", lty = "dashed")
```



Monthly time series

Read the Monte Cimone CO_2 series from file `mtcimone.txt`:

```
cimone <- scan("mtcimone.txt")
```

Transform the series in an `ts` object and plot

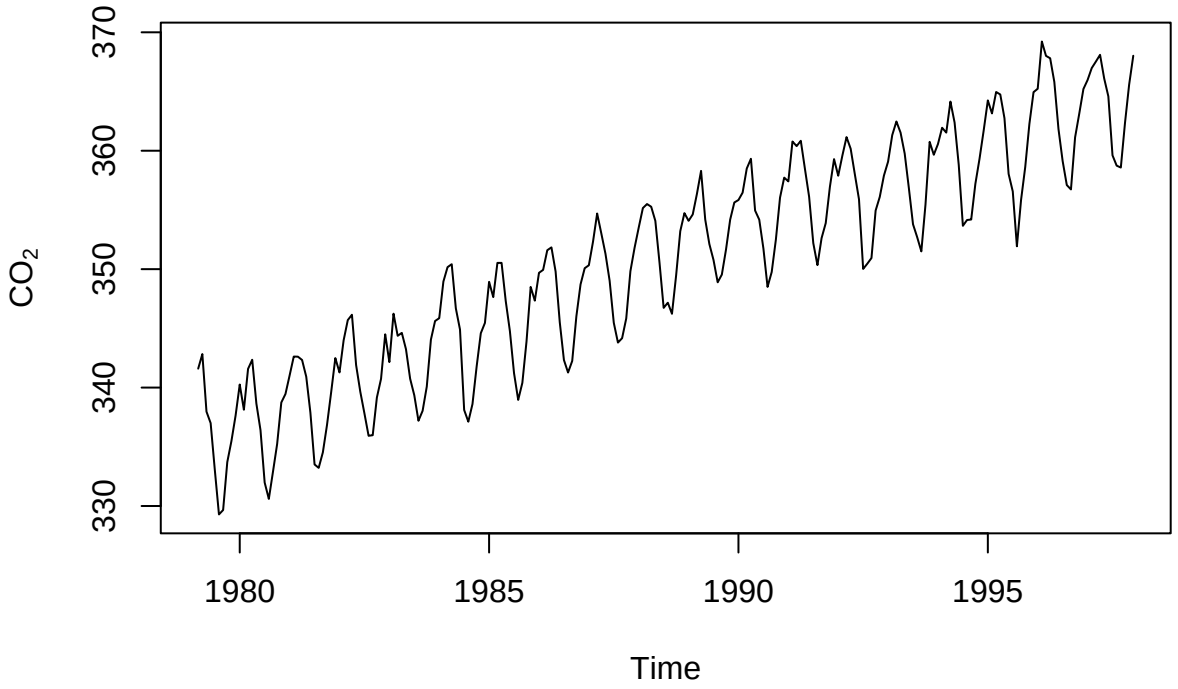
```
cimone.ts <- ts(cimone, start = c(1979, 3), frequency = 12)
head(cimone.ts)
```

```
##         Mar    Apr    May    Jun    Jul    Aug
## 1979 341.60 342.83 337.97 337.00 333.06 329.29
```

```
head(time(cimone.ts))
```

```
##           Mar      Apr      May      Jun      Jul      Aug
## 1979 1979.167 1979.250 1979.333 1979.417 1979.500 1979.583
```

```
plot(cimone.ts, ylab = expression(CO[2]))
```



Multivariate time series

Read the data in the spreadsheet `fish_data.csv`:

```
fish.data <- read.csv("fish_data.csv", header = TRUE)
```

Visualize the data:

```
head(fish.data)
```

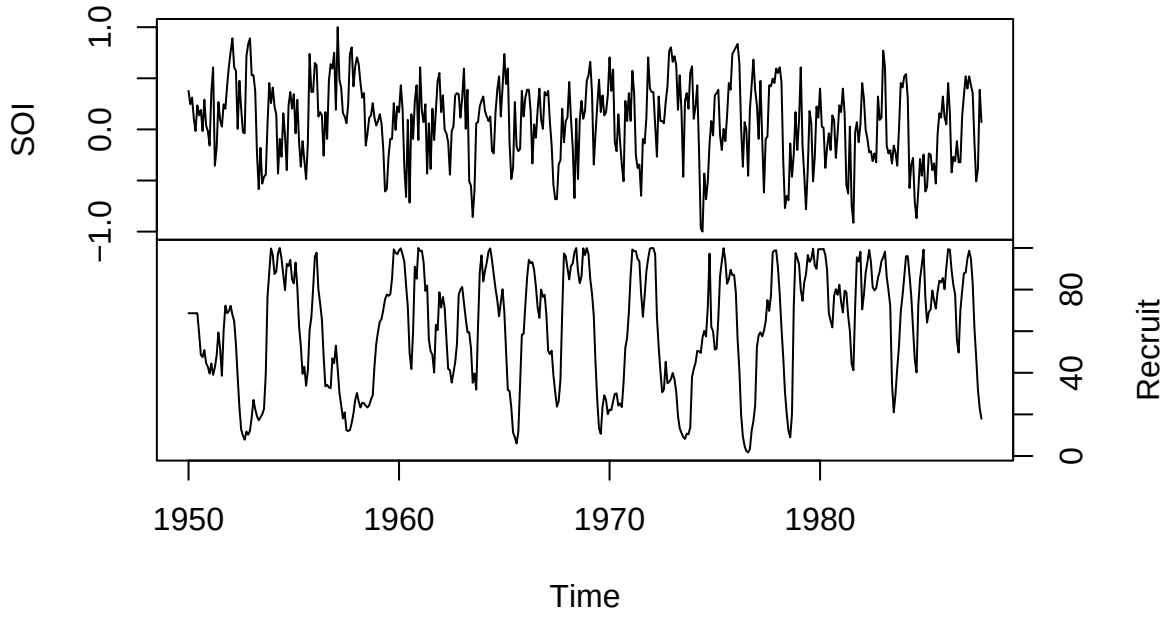
```
##   Year Month    SOI Recruit
## 1 1950     1  0.377   68.63
## 2 1950     2  0.246   68.63
## 3 1950     3  0.311   68.63
## 4 1950     4  0.104   68.63
## 5 1950     5 -0.016   68.63
## 6 1950     6  0.235   68.63
```

```
summary(fish.data)
```

```
##       Year          Month            SOI              Recruit
##  Min.   :1950   Min.   : 1.00   Min.   :-1.00000   Min.   :  1.72
##  1st Qu.:1959   1st Qu.: 3.00   1st Qu.:-0.18000   1st Qu.: 39.62
##  Median :1968   Median : 6.00   Median : 0.11500   Median : 68.63
##  Mean   :1968   Mean   : 6.47   Mean   : 0.08004   Mean   : 62.26
##  3rd Qu.:1978   3rd Qu.: 9.00   3rd Qu.: 0.36600   3rd Qu.: 86.85
##  Max.   :1987   Max.   :12.00   Max.   : 1.00000   Max.   :100.00
```

Bivariate time series plot:

```
fish.ts <- ts(fish.data[, -c(1, 2)], start = 1950, frequency = 12)
plot(fish.ts, main = "Fish data", yax.flip = TRUE)
```



Daily time series: the xts package

We shall consider a dataset on daily Precipitation in De Bilt Amount is in 0.1 mm (-1 for <0.05 mm)

```
debilt<-read.table("KNMI_20160831.txt",header = FALSE,skip=12,sep=",",na.strings = "")
names(debilt)<-c("STN","YYYYMMDD","RH")
head(debilt)
```

```
##   STN YYYYMMDD RH
## 1 260 19010101 NA
## 2 260 19010102 NA
## 3 260 19010103 NA
## 4 260 19010104 NA
## 5 260 19010105 NA
## 6 260 19010106 NA
```

We transfor the censored data

```
censored<-debilt$RH == -1
debilt$RH[censored]<-0.5
```

The data have a daily frequency that cannot be handle by class `ts`. We consider, instead, function `xts` ('eXtensible Time Series') available in the library of the same name:

```
install.packages("xts") ## only the first time!
```

We can now load the library:

```
library(xts)
```

```
## Loading required package: zoo
```

```
##
```

```
## Attaching package: 'zoo'
```

```
## The following objects are masked from 'package:base':
##
##     as.Date, as.Date.numeric
```

The first step is to specify the time series dates as an `Date` object:

```
days <- seq(from = as.Date("1901-01-01"), length.out = nrow(debilt), by = "day")
head(days)
```

```
## [1] "1901-01-01" "1901-01-02" "1901-01-03" "1901-01-04" "1901-01-05"
## [6] "1901-01-06"
```

or

```
days<-as.Date(as.character(debilt$YYYYMMDD),format="%Y%m%d")
```

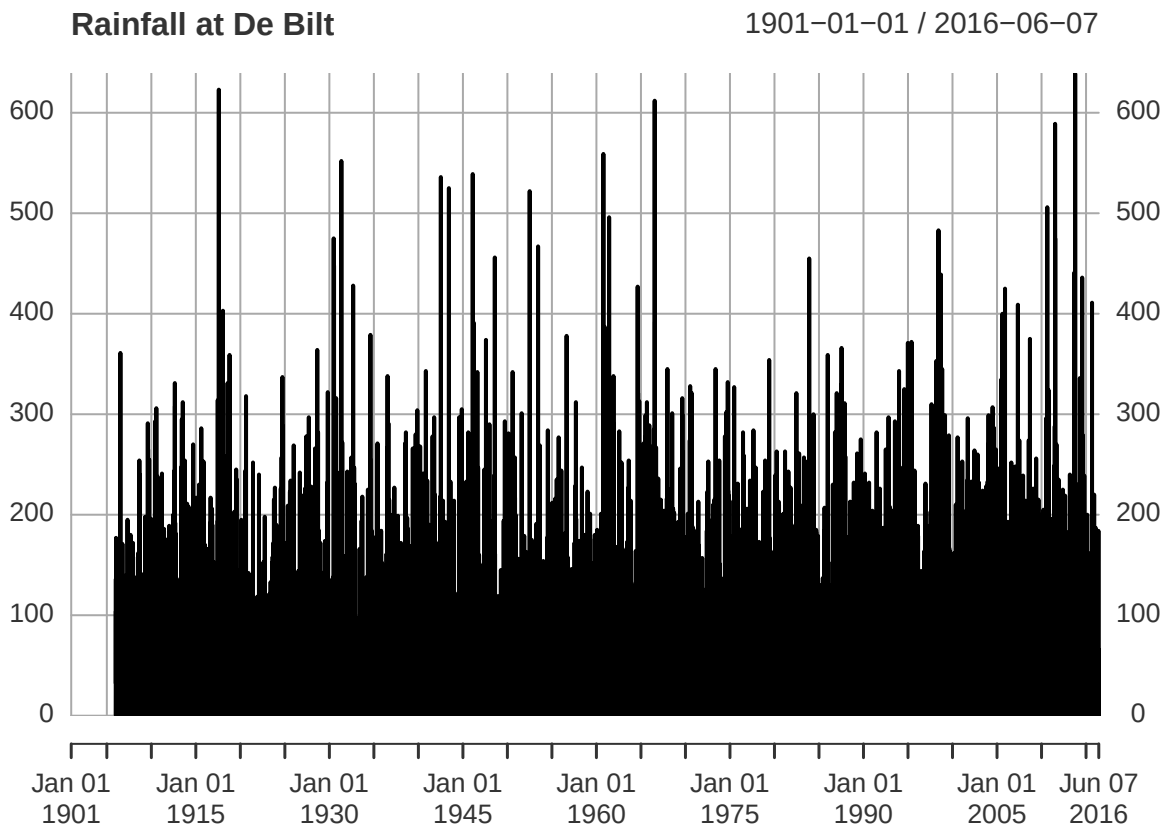
Then, we can set the `xts` object:

```
rainfall <-  xts(debilt$RH, order.by = days, frequency = 7)
```

The `by` argument can be used also to create series with period week, month, etc.).

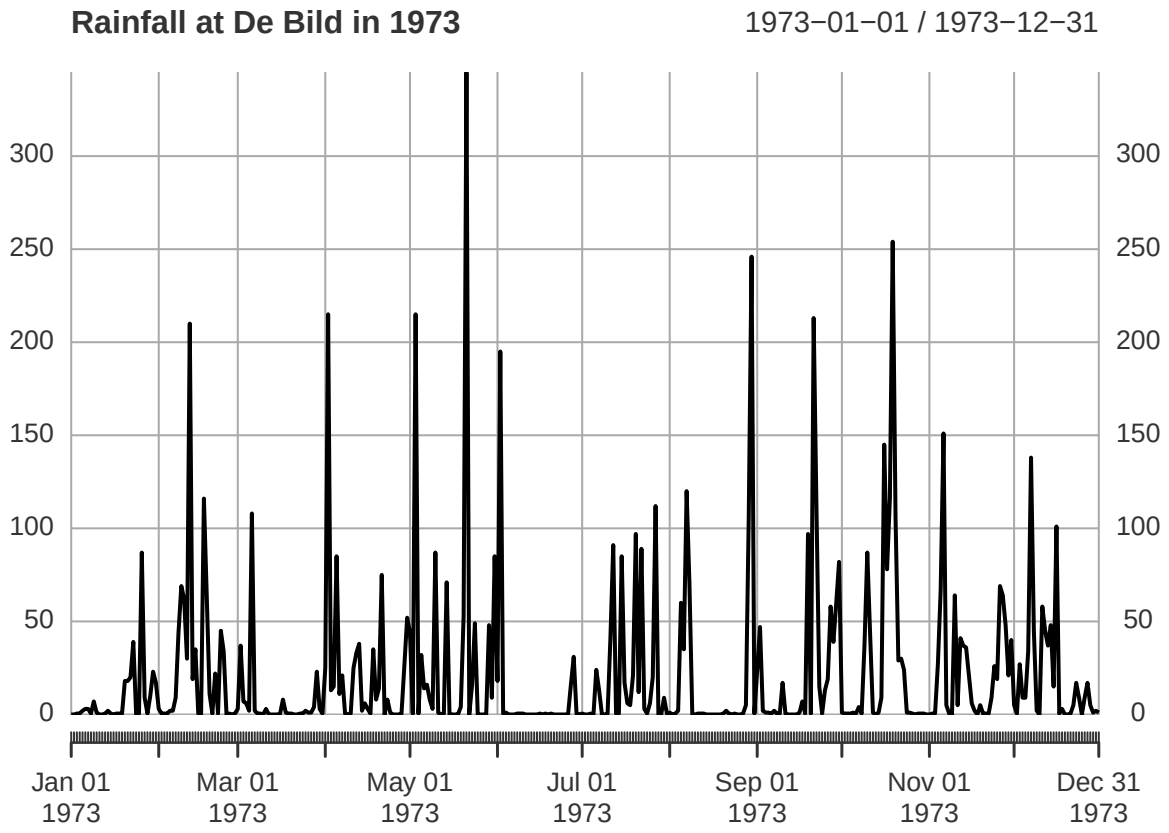
Plot of the time series:

```
plot(rainfall, main = "Rainfall at De Bilt")
```



Function `window` can be used as in `ts` for plotting subseries, for example:

```
plot(window(rainfall, start = as.Date("1973-01-01"), end = as.Date("1973-12-31")), main = "Rainfall at
```



Extracting the entire month of July 1973 - without having to manually identify the index positions or match the underlying index type

```
rainfall['1973-07']
```

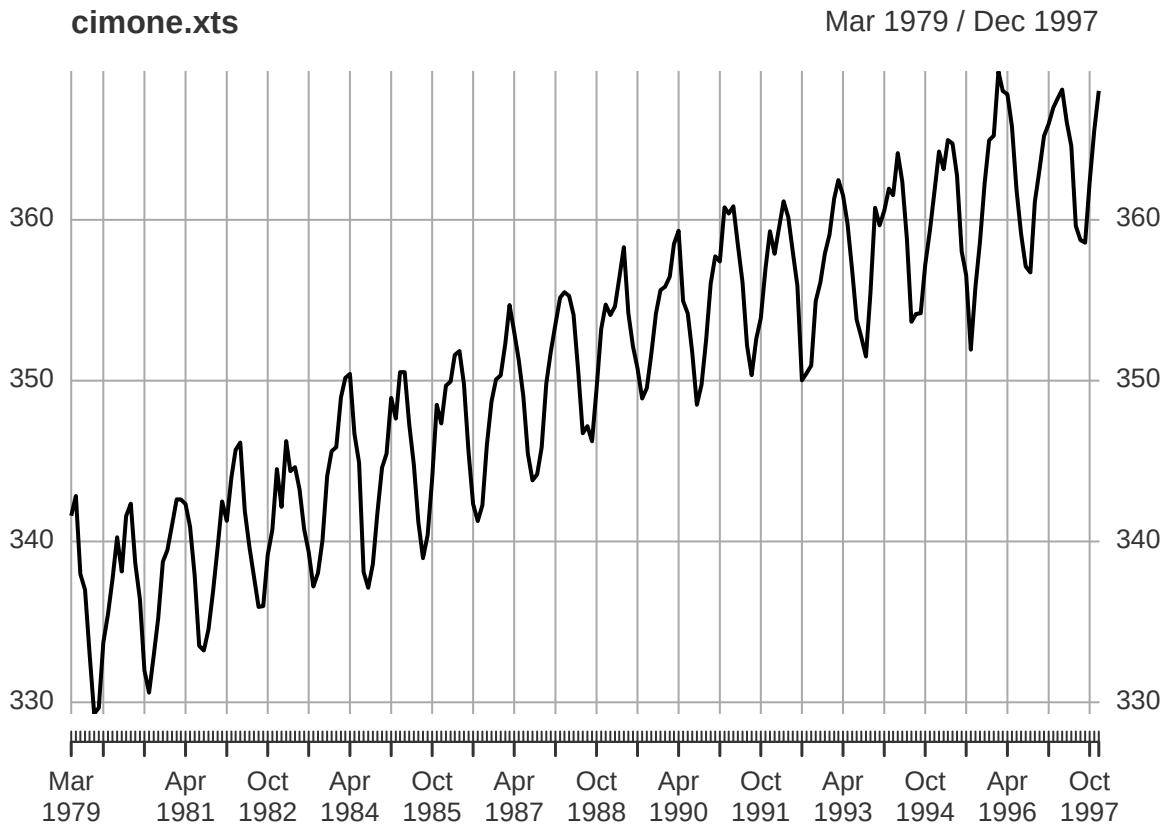
```
##             [,1]
## 1973-07-01   0.5
## 1973-07-02   0.0
## 1973-07-03   0.0
## 1973-07-04   0.5
## 1973-07-05   0.0
## 1973-07-06  24.0
## 1973-07-07  13.0
## 1973-07-08   0.0
## 1973-07-09   0.0
## 1973-07-10   0.5
## 1973-07-11  41.0
## 1973-07-12  91.0
## 1973-07-13   0.5
## 1973-07-14   0.5
## 1973-07-15  85.0
## 1973-07-16  17.0
## 1973-07-17   6.0
## 1973-07-18   5.0
## 1973-07-19  21.0
## 1973-07-20  97.0
## 1973-07-21  12.0
## 1973-07-22  89.0
```

```
## 1973-07-23   3.0
## 1973-07-24   0.5
## 1973-07-25   6.0
## 1973-07-26  20.0
## 1973-07-27 112.0
## 1973-07-28   0.0
## 1973-07-29   0.0
## 1973-07-30   9.0
## 1973-07-31   0.5
```

Flexibility of `xts`

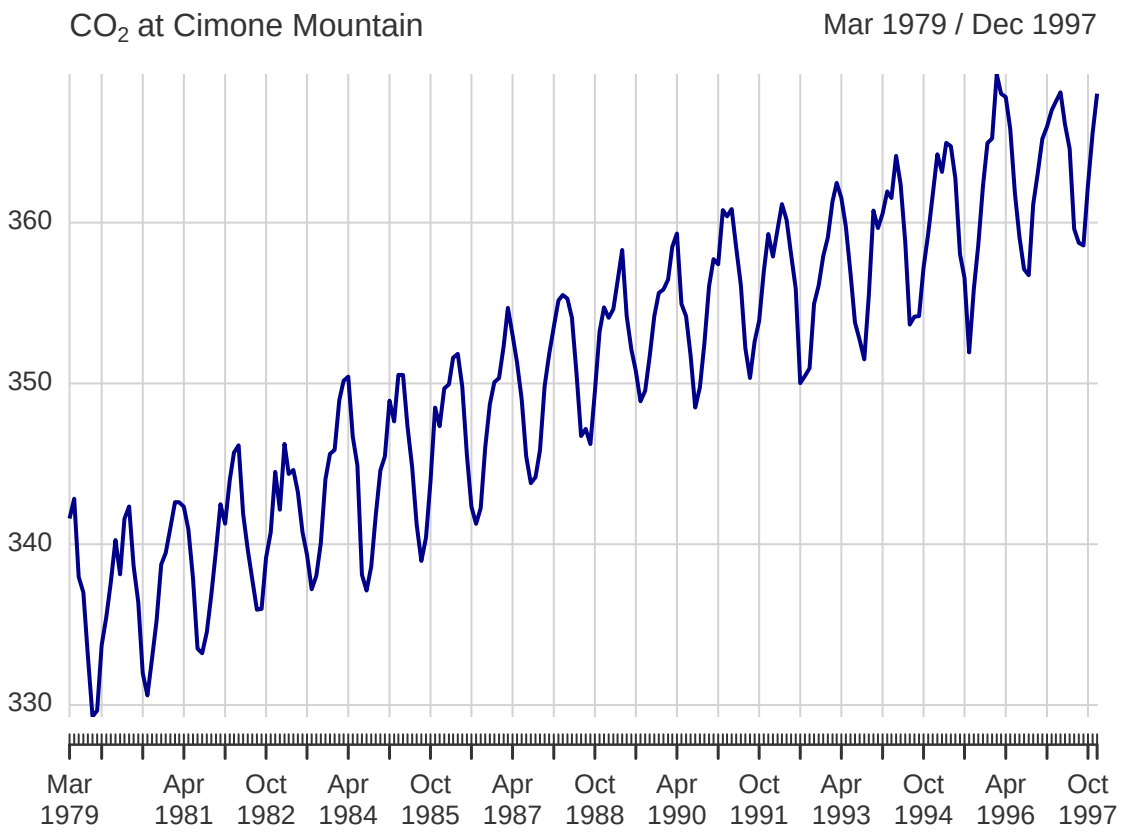
We can transform previously constructed `ts` series in `xts` using function `as.xts`:

```
cimone.xts <- as.xts(cimone.ts)
plot(cimone.xts)
```



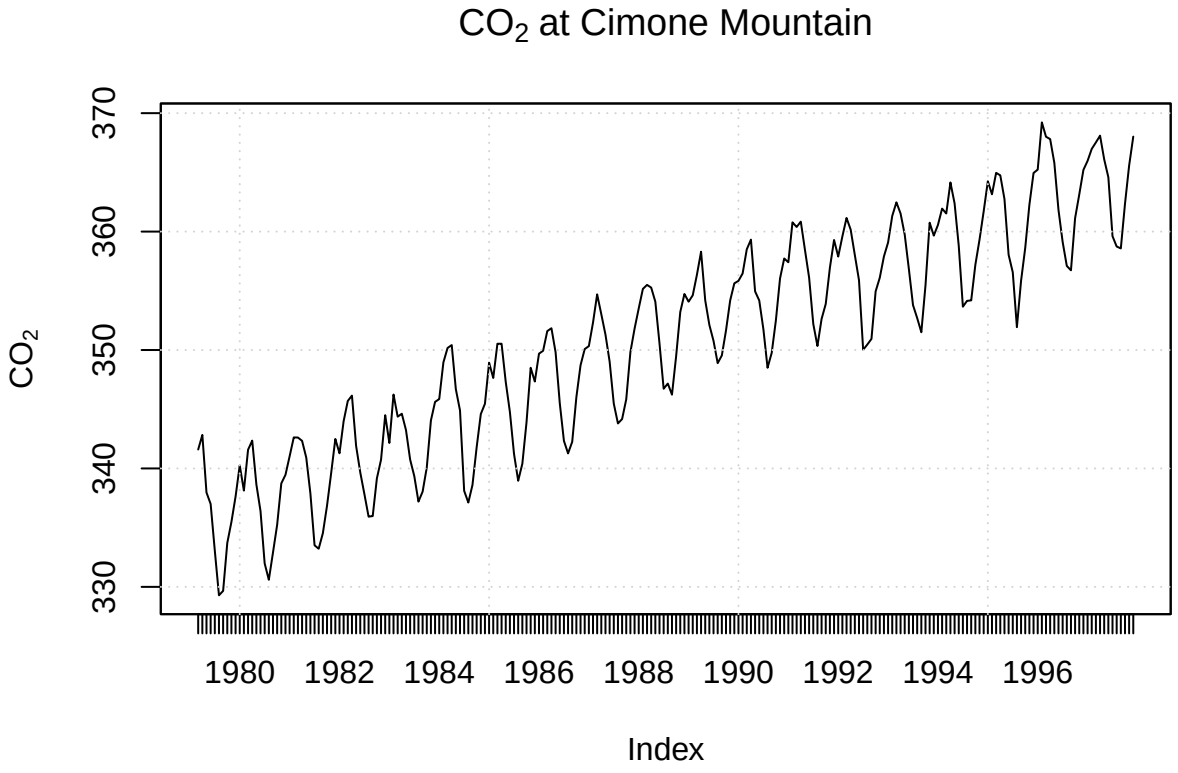
... or with a better title and other features.

```
plot(cimone.xts,main = bquote(CO[2] ~ "at Cimone Mountain"),col = "darkblue",
     grid.col = "lightgrey",yaxis.right = FALSE)
```



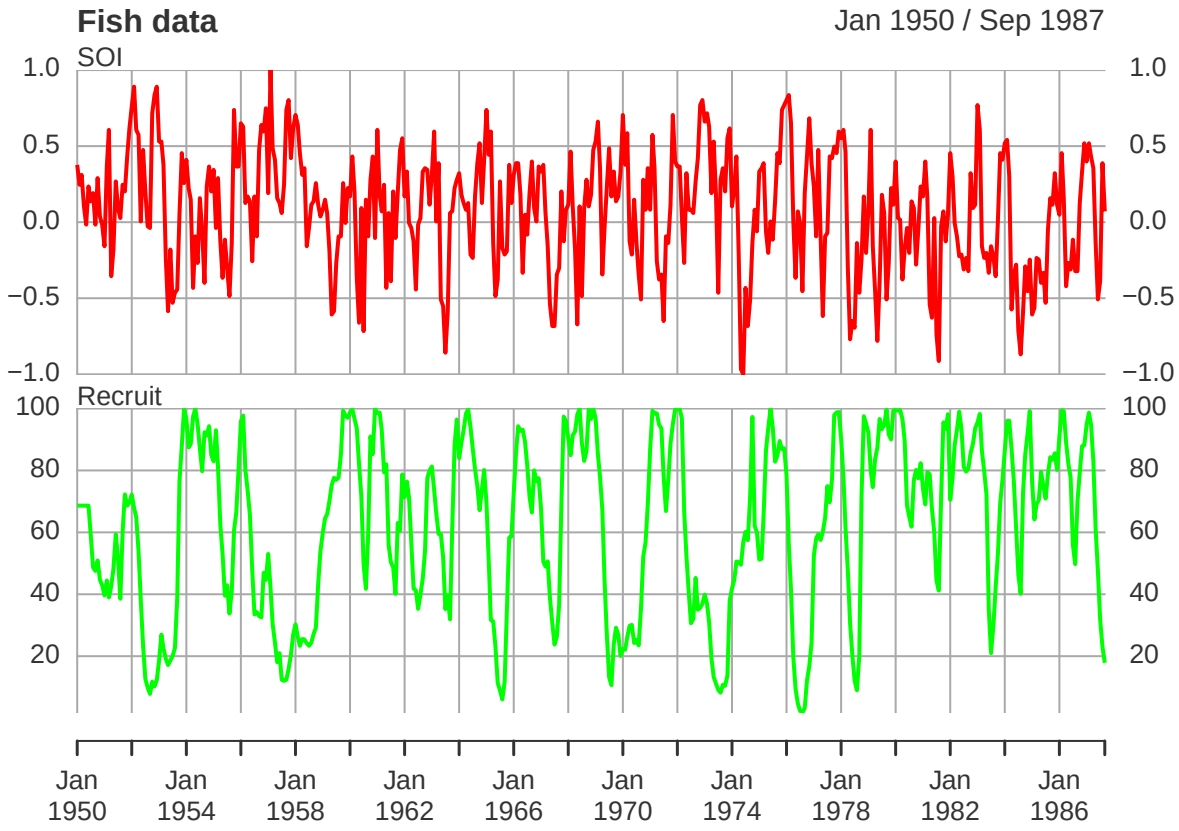
Labels on xy-axes are difficult to manage. Here a customized plot using `zoo` package.

```
zoo::plot.zoo(cimone.xts,main = bquote(CO[2] ~ "at Cimone Mountain"),ylab=bquote(CO[2]))
grid()
```

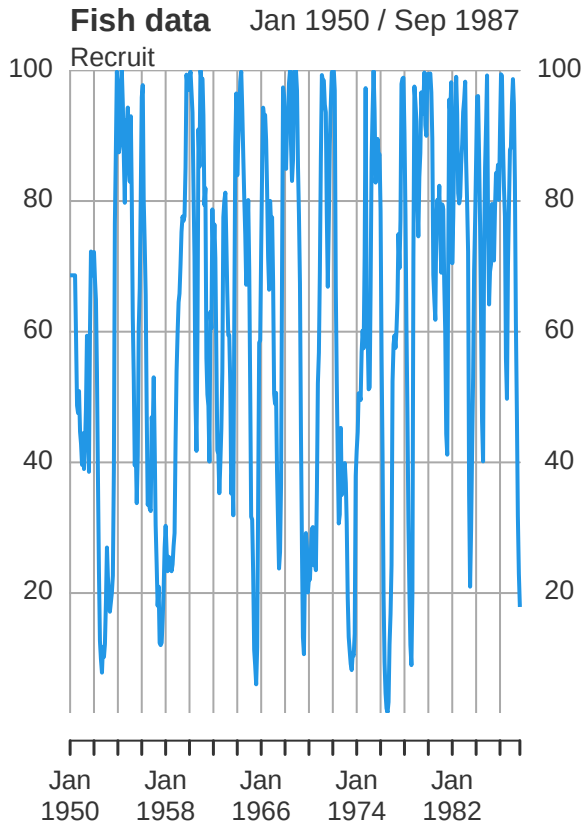


Multiple time series

```
fish.xts <- as.xts(fish.ts)
plot(fish.xts,multi.panel=TRUE,yaxis.same=FALSE,main = "Fish data", col=c("red", "green"))
```




```
layout(matrix(1:2, 1, 2))
plot(fish.xts,multi.panel=1,yaxis.same=FALSE,main = "Fish data", col=c(2, 4))
```



Useful functions

Package `xts` contains several useful functions for computations with time series, some examples:

```
nyears(cimone.xts)
```

```
## [1] 19
```

```
nmonths(cimone.xts)
```

```
## [1] 226
```

```
cimone.yearly <- apply.yearly(cimone.xts, mean, na.rm = TRUE)
```

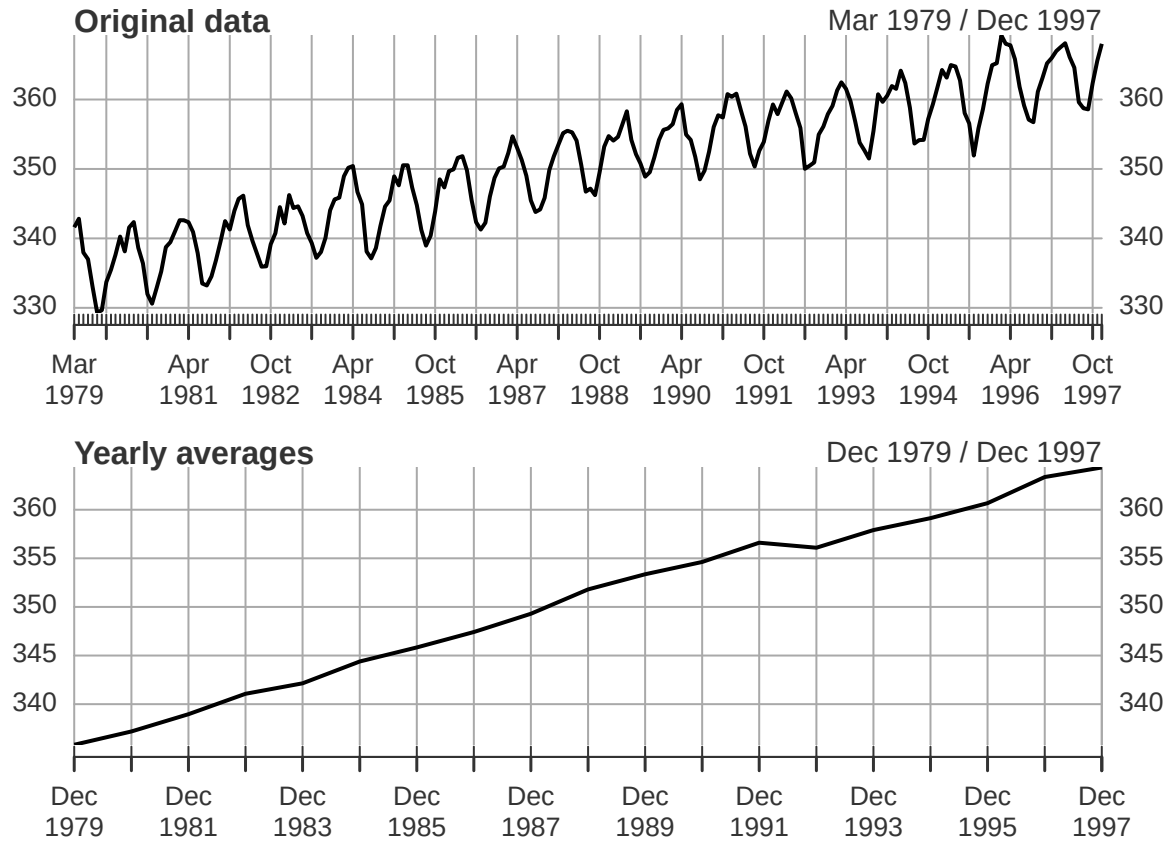
```
## NOTE: `apply.yearly(..., FUN = mean)` operates by column, unlike other math
##   functions (e.g. median, sum, var, sd). Please use `FUN = colMeans` instead,
##   and use `FUN = function(x) mean(x)` to take the mean of all columns. Set
##   `options(xts.message.period.apply.mean = FALSE)` to suppress this message.
```

```
head(cimone.yearly)
```

```
##              [,1]
## Dec 1979 335.8230
## Dec 1980 337.1908
## Dec 1981 338.9667
## Dec 1982 341.0675
## Dec 1983 342.1425
## Dec 1984 344.3883
```

Compare the original series with the series of the quarterly averages

```
par(mfrow = c(2, 1))
plot(cimone.xts, main = "Original data")
plot(cimone.yearly, main = "Yearly averages")
```



Other functions: `apply.daily`,`apply.weekly`, `apply.monthly`, `apply.quarterly`.

A more general function is `enppoints`

```
ep<-endpoints(cimone.xts,on="years")
yh<-period.apply(cimone.xts, INDEX = ep, FUN = colMeans,na.rm=TRUE)
head(yh)
```

```
##              [,1]
## Dec 1979 335.8230
## Dec 1980 337.1908
## Dec 1981 338.9667
## Dec 1982 341.0675
## Dec 1983 342.1425
## Dec 1984 344.3883
```

Valid values for the argument `on` include: “us” (microseconds), “microseconds”, “ms” (milliseconds), “milliseconds”, “secs” (seconds), “seconds”, “mins” (minutes), “minutes”, “hours”, “days”, “weeks”, “months”, “quarters”, and “years”.